DOCUCHAT AI SYSTEM MANUAL • RELEASE 2.0

# End-to-End Execution & File-by-File Order Guide

A comprehensive, chronological engineering walkthrough of the entire DocuChat production RAG platform: what happens step by step when you upload a document, which server process runs, which background worker is dispatched, which AI model executes, how hybrid retrieval fuses with cross-encoder reranking, how agentic tools are evaluated, and the exact role of all 35 source code files in the repository.

| 8 | 4 | 35+ | 100% |
|---|---|---|---|
| SERVERS & SERVICES | AI / ML MODELS | SOURCE CODE FILES | PRIVATE & LOCAL RAG |

## GUIDE CONTENTS & STRUCTURE


1. System Topology: Servers, Services & AI Models **Sec 1**
2. Visual End-to-End Pipeline Diagrams **Sec 2**
3. Step-by-Step Order: When You Upload a Document **Sec 3**
4. Step-by-Step Order: When You Ask a Question / Chat **Sec 4**
5. Complete File-by-File Codebase Catalog **Sec 5**
6. Data Schemas, Resiliency & Quick Reference **Sec 6**


**Target Repository:** DocuChat AI • Branch: `main` • Python 3.11+ / React 18 / PostgreSQL 16 / Qdrant Updated: October 2026 • Production Architecture

# 1. System Topology: Servers, Services & AI Models

DocuChat AI operates as a unified, enterprise-grade multi-tier architecture consisting of **8 distinct server processes and services** and **4 machine learning models** working in complete orchestration.

## 1.1 The 8 Servers & Services in Execution

| SERVER / SERVICE | PORT | TECHNOLOGY / IMAGE | HOST / URL | PRIMARY ROLE & OPERATIONAL LIFECYCLE |
|---|---|---|---|---|
| **1. Frontend Web App** | `3000` | React 18 SPA (Vite / Dev / Nginx) | `http://localhost:3000` | Interactive user interface, file drag-and-drop zone, SSE stream consumer, bidirectional WebSocket chat client, and PDF.js canvas bounding box highlighter. |
| **2. Backend API Gateway** | `5000` | FastAPI / Uvicorn (Python 3.11+) | `http://localhost:5000` | Core API gateway. Handles multi-tenant JWT auth, document ingress staging, query routing, SSE token streaming, safe tool execution, and database sessions. |
| **3. Background Ingestion Worker** | `Internal` | Python Worker ( `backend.workers.ingest_worker` ) | `Container / Subprocess` | Decoupled asynchronous worker. Consumes jobs from Redis / task queue, runs Docling extraction, structure chunking, embeddings generation, and vector indexing without blocking the API. |
| **4. Docling Layout Engine** | `5001` | `quay.io/docling-project/docling-serve` | `http://localhost:5001` | IBM Docling layout vision container. Extracts headings, tables, reading order, and exact PDF bounding box coordinates ( `x0, y0, x1, y1` ). Has PyMuPDF fallback. |
| **5. Local LLM Service** | `11434` | Ollama Runtime | `http://localhost:11434` | Locally hosted LLM runtime. Serves `qwen2.5:3b` / `qwen2.5:7b` for conversational answering, query expansion, and optional Nomic dense embeddings. |
| **6. Vector Database** | `6333` / `6334` | Qdrant Vector DB Engine | `http://localhost:6333` | Stores dense vector chunks with Cosine distance indexing. Performs sub-millisecond Approximate Nearest Neighbor (ANN) vector search with HNSW graphs. |
| **7. Relational Database** | `5432` | PostgreSQL 16 Alpine / SQLite fallback | `postgres:5432` / `rag_app.db` | Persists multi-tenant users, sessions, documents, ingestion jobs, chat conversations, and full message histories with SQLAlchemy async session pooling. |
| **8. Redis Broker & Cache** | `6379` | Redis 7 Alpine | `redis://localhost:6379/0` | In-memory broker for the ingestion job queue ( `docuchat_ingest_queue` ) and pub/sub message broadcaster for real-time WebSocket progress updates. |

> **Dual Database & Hybrid Storage Strategy:** DocuChat AI connects by default to PostgreSQL 16 via `asyncpg` . If PostgreSQL is offline or running locally without Docker, it seamlessly falls back to local SQLite ( `backend/data/rag_app.db` ) via `aiosqlite` with zero configuration change. Qdrant similarly supports embedded local disk storage in `backend/data/qdrant_storage` .

### 1.2 The 4 Machine Learning Models in the Pipeline

| MODEL NAME | ARCHITECTURE | PARAMETERS / DIM | SERVING ENGINE | EXECUTION STAGE & SPECIFIC RESPONSIBILITY |
|---|---|---|---|---|
| **Qwen 2.5** ( `qwen2.5:3b` / `7b` ) | Decoder-Only Transformer | 3.09B / 7.61B params | Ollama (Port 11434) | **Reasoning, Rewriting & Generation:** Resolves conversational pronouns, generates search query expansions, evaluates agentic tools, and streams grounded answers with citations. |
| **Nomic Embed / BGE-Small** | Encoder Transformer | 768d / 384d normalized | Ollama / FastEmbed ONNX | **Dense Semantic Vectorization:** Converts chunks and queries into dense vectors. Default is `nomic-embed-text:v1.5` (768d) or `BAAI/bge-small-en-v1.5` (384d). |
| **BAAI/bge-reranker-base** | Cross-Encoder Transformer | 278M params | SentenceTransformers (CPU/GPU) | **Precision Relevance Reranking:** Scores candidate chunk pairs `[Query, Passage]` with deep cross-attention, normalized with max-scaling to eliminate irrelevant candidates. |
| **Docling Vision / TableFormer** | Multi-Modal Vision Transformer | Proprietary Docling OCR | Docling-Serve (Port 5001) | **Structural Layout Parsing:** Segments document structures, recognizes complex multi-column layouts, reconstructs markdown tables, and maps exact bounding box coordinates. |

# 2. Visual End-to-End Pipeline Diagrams

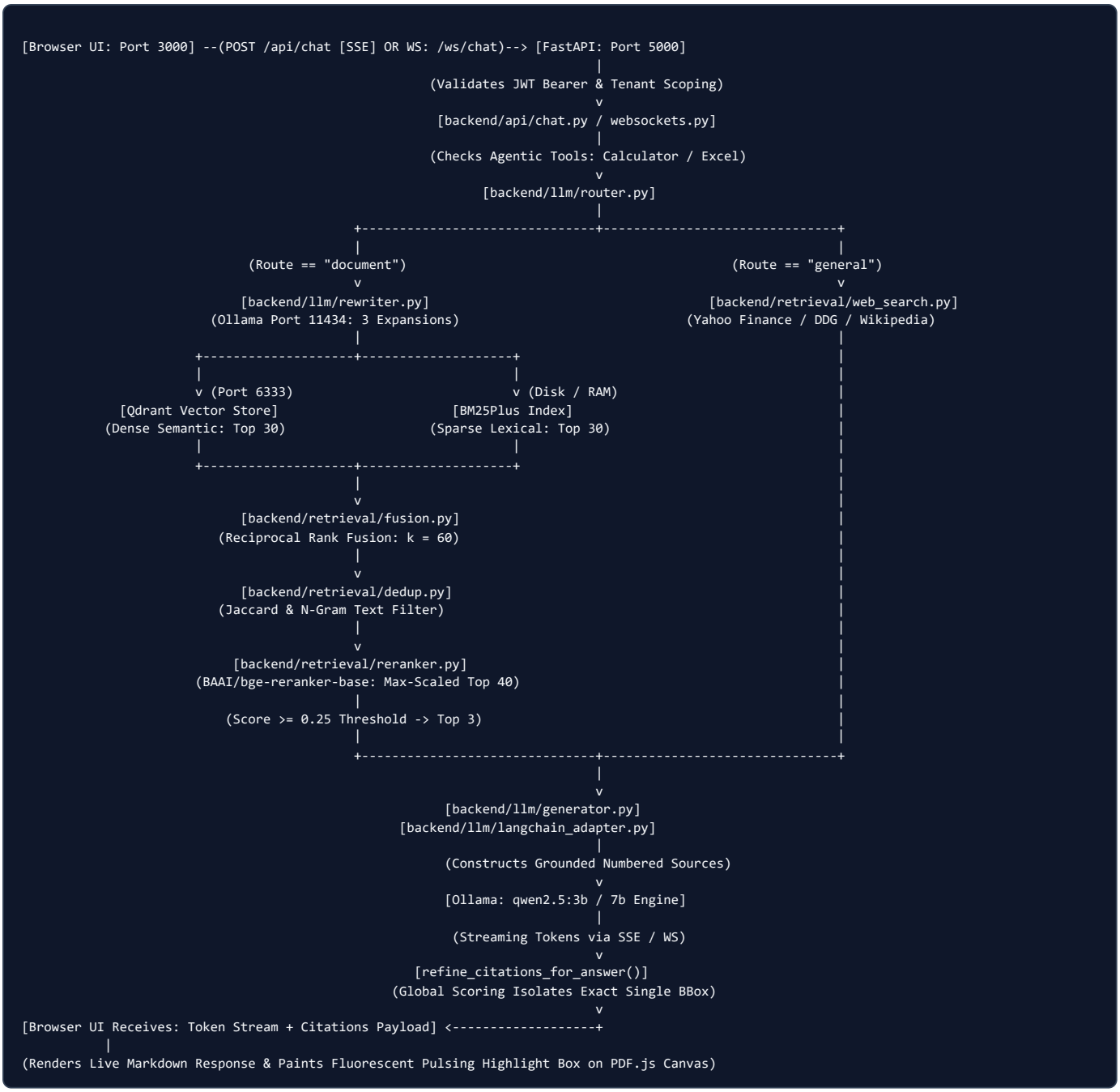
## 2.1 The Document Ingestion, Worker Queue & Multi-Modal Indexing Pipeline

The sequence triggered when a user drops a document (PDF, DOCX, XLSX, TXT, Image) into the DocuChat UI:

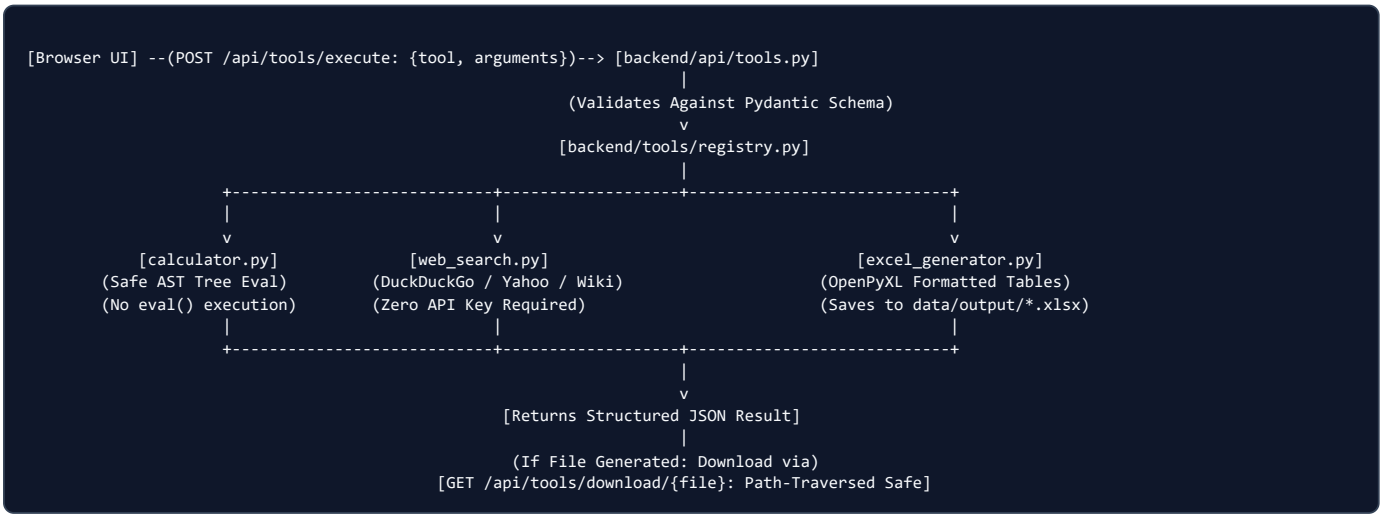



## 2.2 The Multi-Tenant Query, Agentic Routing, Hybrid Retrieval & Grounded Generation Pipeline

The sequence executed when a user submits a question across active documents or in ChatGPT mode:




## 2.3 The Safe Agentic Tool Execution Pipeline



## 3. Step-by-Step Order: When You Upload a Document

When a document is uploaded, DocuChat AI executes an **11-step asynchronous ingestion workflow** designed for non-blocking UI responsiveness, structural layout preservation, and precise provenance tracking.

### 1 Frontend Selection & Multipart HTTP Dispatch

FRONTEND/SRC/APP.JSX PORT 3000 -> 5000

**What Happens:** The user drops or selects a document file in the React UI dropzone. `handleFileUpload(file)` validates the file size, builds a `FormData` multipart body containing `file` and optional authentication token, and sends an asynchronous `POST http://localhost:5000/api/documents`.

**Calling Server:** React Frontend on Port 3000 • **Receiving Server:** FastAPI on Port 5000 • **Active Code:** `App.jsx: lines 450–510`, `services/api.js`.

### 2 FastAPI Upload Validation, Multi-Tenant Auth & Disk Staging

BACKEND/API/DOCUMENTS.PY FASTAPI BACKEND/API/AUTH.PY

**What Happens:** The endpoint `upload_document()` receives the upload. It parses the optional `Authorization: Bearer <token>` to extract `user_id`. It verifies the extension against `ALLOWED_EXTENSIONS` (`.pdf, .docx, .pptx, .xlsx, .txt, .html, .png, .jpg, .jpeg`), generates a UUID (`doc_id = str(uuid.uuid4())`), and streams the file to disk at `backend/data/uploads/{doc_id}_{filename}`.

**Disk Location:** `backend/data/uploads/` • **Active Code:** `documents.py: lines 85–120`.

### 3 Relational DB Row & Ingest Job Record Creation

POSTGRESQL / SQLITE BACKEND/DB/MODELS.PY INGESTJOBMODEL

**What Happens:** Two database records are committed in the relational database: 1. `DocumentModel(id=doc_id, user_id=user_id, filename=filename, status="queued")`, and 2. `IngestJobModel(id=job_id, document_id=doc_id, status="queued", progress=5)`. This ensures persistent job tracking across container restarts.

**Database Records:** `DocumentModel` & `IngestJobModel` • **Active Code:** `documents.py`, `workers/job_queue.py: lines 30–45`.

### 4 Asynchronous Job Enqueuing & WebSocket Subscription

REDIS QUEUE / TASK POOL BACKEND/WORKERS/JOB_QUEUE.PY WEBSOCKET /WS/DOCUMENTS

**What Happens:** `enqueue_document_ingestion()` pushes the job payload onto Redis list `docuchat_ingest_queue`. If Redis is unconfigured or offline, it launches an asynchronous task pool worker. FastAPI immediately returns HTTP 200 with document metadata. The React client immediately opens a live WebSocket connection to `/ws/documents/{doc_id}/progress` to stream real-time percentage updates.

**Protocol:** Redis RPUSH & WebSocket • **Active Code:** `job_queue.py: lines 45–65`, `api/websockets.py: lines 180–260`.

### 5 Format Normalization to Visual PDF

BACKEND/PARSERS/CONVERTER.PY WORKER PROCESS

**What Happens:** The background worker executes `convert_to_pdf(input_path, settings.CONVERTED_DIR)`. If the file is already a PDF, it validates page headers via PyMuPDF (`fitz.open()`). If it is DOCX, PPTX, XLSX, or TXT, it converts it to PDF using headless LibreOffice. If it is an image (PNG, JPG), PyMuPDF builds a PDF page wrapper so the frontend PDF.js canvas can display it identically. Worker broadcasts progress: **15%**.

**Output:** A normalized PDF file path at `backend/data/converted/` • **Active Code:** `converter.py: lines 9–83`.

### 6 Docling Microservice Extraction & Structure Parsing

PORT 5000 -> 5001 DOCLING / TABLEFORMER BACKEND/PARSERS/DOCLING_PARSER.PY

**What Happens:** Worker calls `docling_parser.parse_file()`, making an HTTP `POST http://localhost:5001/v1/convert/file` to the Docling-serve container. Docling executes its deep vision layout model to detect headings, paragraph flows, and tabular cells with exact coordinates: `{"l": left, "t": top, "r": right, "b": bottom, "page_no": page}`.
*Built-in Fault Tolerance:* If Docling is offline, DoclingParser catches the error and executes its native `pymupdf` fallback, extracting blocks and word-level rects with 100% uptime. Worker broadcasts progress: **40%**.

**Output:** `List[ParsedElement]` with text, heading paths, page numbers, and bounding boxes • **Active Code:** `docling_parser.py: lines 45–130`.

### 7 Structure-Aware Token Chunking & Provenance Retention

BACKEND/CHUNKING/STRUCTURE_CHUNKER.PY | IN-PROCESS

**What Happens:** `structure_chunker.chunk_elements()` iterates over `ParsedElement` objects. It groups elements respecting markdown section boundaries ( `# Title > ## Section` ) and table structures until reaching the configured token target ( `CHUNK_TARGET_TOKENS = 400` , max 450, overlap 50). Crucially, it aggregates element bounding boxes and attaches the exact phrase text to each bounding box. Worker broadcasts progress: **65%**.

**Output:** `List[Chunk]` with chunk ID, unified markdown text, page, bboxes, and section metadata • **Active Code:** `structure_chunker.py: lines 35–140` .

### 8 Dense Vector Embedding Generation (AI Model)

NOMIC 768D / BGE 384D | BACKEND/EMBEDDINGS/EMBEDDER.PY

**What Happens:** `embedder.embed_texts(chunk_texts)` is called. If configured for Nomic (default), it invokes Ollama's `/api/embed` endpoint to produce 768-dimensional normalized vectors. If configured for BGE, it loads `BAAI/bge-small-en-v1.5` via FastEmbed ONNX runtime locally for sub-second CPU batch embedding (384 dimensions). Worker broadcasts progress: **80%**.

**Output:** Normalized floating-point vector arrays (dim = 768 or 384) • **Active Code:** `embedder.py: lines 35–80` .

### 9 Vector Storage Upsert & Collection Indexing

PORT 6333 (QDRANT) | BACKEND/RETRIEVAL/VECTOR_STORE.PY

**What Happens:** `vector_store.upsert_chunks()` sends the batch of chunk vectors to Qdrant collection `doc_chunks` . Every point stores: vector, ID, and payload ( `doc_id` , `chunk_id` , `page` , `text` , `bboxes` , `heading_path` ). Qdrant indexes points into an in-memory HNSW graph for fast Cosine similarity retrieval. Worker broadcasts progress: **90%**.

**Target DB:** Qdrant Vector DB on Port 6333 • **Active Code:** `vector_store.py: lines 65–110` .

### 10 BM25Plus Lexical Inverted Indexing

BACKEND/RETRIEVAL/BM25.PY | DISK: DATA/BM25

**What Happens:** `bm25_index.build_index_for_document(doc_id, chunks)` tokenizes chunks into stemmed lowercase n-grams, computes inverse document frequencies (IDF), and serializes the index to disk at `backend/data/bm25/{doc_id}.json` . Using the BM25Plus algorithm prevents score clipping on long passages. Worker broadcasts progress: **95%**.

**Disk File:** `backend/data/bm25/{doc_id}.json` • **Active Code:** `bm25.py: lines 45–120` .

### 11 Relational DB Finalization & Real-Time WebSocket Completion

POSTGRESQL / SQLITE | WEBSOCKET EVENT | BACKEND/WORKERS/INGEST_WORKER.PY

**What Happens:** Worker updates `DocumentModel(id=doc_id, status="ready", chunk_count=len(chunks), page_count=pages)` and sets `IngestJobModel(status="completed", progress=100)` . It publishes the final completion event over Redis pub/sub and local WebSocket channels. The React client displays the document as ready, switches the badge to green, and mounts the document into the PDF canvas viewer.

**Active Code:** `ingest_worker.py: lines 140–190` , `App.jsx: lines 510–560` .

## 4. Step-by-Step Order: When You Ask a Question / Chat

When a user types a prompt and presses Enter, DocuChat AI executes an **11-step inference, hybrid retrieval, and grounded generation pipeline** with strict provenance guarantees.

### 1 User Message Input & SSE / WebSocket Dispatch

FRONTEND/SRC/COMPONENTS/CHAT/CHATINPUT.JSX · PORT 3000 -> 5000 · SSE / WS

**What Happens:** The user submits a prompt. The frontend checks connection preferences: it either sends an HTTP POST `/api/chat` expecting a Server-Sent Events (SSE) stream or dispatches a JSON packet over the existing `/ws/chat` WebSocket channel containing: `{message, conversation_id, active_doc_ids}`.

**Active Code:** `ChatInput.jsx`, `services/api.js`, `services/websocket.js`.

### 2 Multi-Tenant Session Validation & Context Window Assembly

POSTGRESQL / SQLITE · BACKEND/API/CHAT.PY · BACKEND/API/WEBSOCKETS.PY

**What Happens:** Backend validates JWT bearer token to verify user ownership. It looks up or creates the `ConversationModel`, saves the user message as a new `MessageModel(role="user", content=message)`, and queries the database for the last 6 conversation turns to maintain conversational continuity.

**Active Code:** `chat.py: lines 35–75`, `websockets.py: lines 60–110`.

### 3 Agentic Tool Intent Evaluation

AGENTIC TOOLS · BACKEND/TOOLS/REGISTRY.PY · BACKEND/LLM/ROUTER.PY

**What Happens:** Before performing full document retrieval, the system evaluates if the prompt requires a vetted tool: 1. Arithmetic or calculations → `calculator` (safe AST parser, zero eval), 2. Tabular generation requests (e.g. "export as Excel sheet") → `export_excel` (OpenPyXL generator), 3. Real-time market, movie, or weather queries → `web_search`. If a standalone tool matches, it executes safely within sandbox limits.

**Active Code:** `registry.py: lines 50–90`, `tools.py: lines 20–40`.

### 4 Intent Routing: Document Mode vs ChatGPT Real-World Mode

BACKEND/LLM/ROUTER.PY · IN-PROCESS

**What Happens:** `query_router.route_query()` inspects the active document scope and question text: 1. If no documents exist or `active_doc_ids` is empty → routes to `"general"` mode (ChatGPT real-world mode), 2. If documents are active and query is a conversational greeting (e.g., "hello", "who are you") → routes to `"general"`, 3. If documents are active and query asks for facts → routes to `"document"` mode to enforce strict document grounding.

**Output:** `(route_type, badge_label)` • **Active Code:** `router.py: lines 15–44`.

### 5 Multi-Query Rewriting & Pronoun Disambiguation (Server Call & AI Model)

QWEN 2.5 (11434) · BACKEND/LLM/REWRITER.PY

**What Happens:** If in `"document"` mode, `query_rewriter.transform_query()` analyzes the question and chat history. It prompts Ollama to resolve ambiguous references (e.g., converting "What were its results?" to "What were the BLEU score results of Transformer base model?"). It also triggers specialized rule expansions for tables, summaries, and financial reports, returning 3 search variants: `[standalone_query, alt_1, alt_2]`.

**Active Code:** `rewriter.py: lines 20–95`.

### 6 Parallel Hybrid Retrieval: Dense Vectors (Qdrant) + Sparse Lexical (BM25Plus)

QDRANT PORT 6333 · BACKEND/RETRIEVAL/PIPELINE.PY · DISK: DATA/BM25

**What Happens:** `retrieval_pipeline.retrieve()` executes parallel async retrieval for each generated query variant: 1. **Dense Vector Search:** Embeds query via `embedder.embed_query()` and searches Qdrant (Top 30 by Cosine distance), 2. **Sparse Lexical Search:** Evaluates query against the BM25Plus index on disk (Top 30 by keyword frequency). This guarantees exact term matching (e.g. acronyms, IDs, dates) and semantic matching.

**Output:** Parallel ranking lists from Qdrant and BM25Plus • **Active Code:** `pipeline.py: lines 40–65`.

### 7 Reciprocal Rank Fusion (RRF) & Semantic Deduplication

BACKEND/RETRIEVAL/FUSION.PY BACKEND/RETRIEVAL/DEDUP.PY

**What Happens:** Candidate lists are combined using Reciprocal Rank Fusion: $RRF(d) = \sum \frac{1}{60 + rank(d)}$. This unifies dissimilar vector and BM25 scoring scales into a single fair ranking. Then, `deduplicate_chunks()` calculates Jaccard word similarity and character 3-gram overlap, removing redundant or overlapping text chunks with similarity $> 0.88$.

**Active Code:** `fusion.py: lines 12–55`, `dedup.py: lines 10–45`.

### 8 Cross-Encoder Deep Reranking & Max-Scaling Normalization (AI Model)

BAAI/BGE-RERANKER-BASE BACKEND/RETRIEVAL/RERANKER.PY

**What Happens:** The top 40 candidate chunks are passed to `reranker.rerank()`. `BAAI/bge-reranker-base` evaluates the combined pair `[Query, Chunk Text]` using full cross-attention. It applies **Max-Scaling Normalization** ( `raw / max_s` ) so that secondary relevant passages are not artificially suppressed by sigmoid flattening. Passages below `SCORE_THRESHOLD = 0.25` are dropped. The top 3–5 highest-ranking passages are selected.

**Active Code:** `reranker.py: lines 25–75`.

### 9 Grounded Prompt Assembly, LangChain Orchestration & Token Streaming (AI Model)

QWEN 2.5 (11434) BACKEND/LLM/GENERATOR.PY BACKEND/LLM/LANGCHAIN_ADAPTER.PY

**What Happens:** `grounded_generator` formats the retrieved passages into numbered context blocks `[1] (Page P): text` with strict anti-hallucination system directives. It invokes Ollama ( `qwen2.5:3b` / `7b` ) via native streaming or the LangChain ChatOllama adapter. Tokens stream character-by-character to the client UI as SSE events ( `event: token` ) or WebSocket messages.

**Active Code:** `generator.py: lines 20–110`, `langchain_adapter.py: lines 70–160`.

### 10 Real-Time Live Web Knowledge Fallback

BACKEND/RETRIEVAL/WEB_SEARCH.PY YAHOO / DDG / WIKI

**What Happens:** If the model concludes that the document lacks the requested answer (e.g. “This is not mentioned in the document”) or if routed in general ChatGPT mode, the system seamlessly queries `live_web_search`. It hits Yahoo Finance API for real-time tickers, DuckDuckGo Lite for current news/movies, and Wikipedia REST API for sports/facts with zero API key dependencies.

**Active Code:** `web_search.py: lines 25–190`.

### 11 Global Best-Box Refinement & Synchronized Canvas Highlighting

BACKEND/LLM/GENERATOR.PY FRONTEND/SRC/COMPONENTS/PDFVIEWER/PDFVIEWERDRAWER.JSX

**What Happens:** Once generation completes, `refine_citations_for_answer()` performs global n-gram intersection scoring between generated answer tokens and chunk bounding box text. It isolates the **single winning bounding box** containing the answer, preventing the legacy “blue screen” multi-box clutter bug. The citation event transmits `{page, bboxes: [winning_box]}` to the React viewer, triggering a high-contrast pulsing glow on the PDF.js HTML5 canvas.

**Active Code:** `generator.py: lines 135–220`, `PdfViewerDrawer.jsx`.

# 5. Complete File-by-File Codebase Catalog

An exhaustive architectural breakdown of all 35 source code files in the DocuChat AI repository, grouped by architectural subsystem with their exact roles, callers, and callees.

## 5.1 API Gateway & Application Layer

**`backend/main.py`** — FASTAPI ENTRYPOINT

Initializes the FastAPI application, configures CORS middleware, registers request ID correlation headers, initializes database schemas via `init_db()`, seeds default demo users, ensures Qdrant collection existence, and mounts all REST routers and WebSocket endpoints.

• Called By: Uvicorn / Docker CMD • Calls: config.py, db/session.py, retrieval/vector_store.py, api/ routers

**`backend/config.py`** — CONFIGURATION ENGINE

Centralized Pydantic settings singleton ( `Settings` ). Defines network ports, model identifiers ( `LLM_MODEL="qwen2.5:3b"` , `EMBEDDING_MODEL="nomic-embed-text:v1.5"` ), retrieval tuning thresholds ( `RETRIEVAL_TOP_K=30` , `RRF_K=60` , `FINAL_TOP_K=3` ), directory paths, and auto-detects embedding dimensions (768 vs 384).

• Called By: Every module across backend • Calls: pydantic_settings

**`backend/api/documents.py`** — DOCUMENT INGESTION API

REST controller for document lifecycle. Validates uploads, checks file types, generates UUIDs, stages bytes to disk, persists database records, and dispatches asynchronous ingestion to the job queue. Also provides document deletion and raw file serving.

• Called By: React UI (App.jsx) • Calls: config.py, db/models.py, workers/job_queue.py

**`backend/api/chat.py`** — STREAMING CHAT API

Server-Sent Events (SSE) streaming chat endpoint ( `POST /api/chat` ). Handles multi-tenant JWT auth, session lookups, query routing, conversational history persistence, and streams real-time answer tokens and refined citation bounding boxes.

• Called By: React UI (ChatInput.jsx) • Calls: router.py, rewriter.py, pipeline.py, generator.py, db/session.py

**`backend/api/conversations.py`** — CHAT SESSIONS API

CRUD endpoints for conversation sessions ( `/api/conversations` ). Lists user conversations, retrieves full chronological message histories, updates conversation titles, and executes cascading session deletions.

• Called By: React UI (Sidebar.jsx, HistoryModal.jsx) • Calls: db/models.py, db/session.py

**`backend/api/auth.py`** — MULTI-TENANT AUTH ENGINE

Enterprise user authentication and session management. Implements Argon2id password hashing with PBKDF2-HMAC-SHA256 fallback, signed JWT bearer tokens with 24-hour expiration, Role-Based Access Control (RBAC: `admin` / `user` ), and default seeded demo accounts.

• Called By: main.py, chat.py, documents.py • Calls: argon2, jwt, db/models.py

**`backend/api/websockets.py`** — PRODUCTION WEBSOCKETS

Bidirectional real-time WebSocket endpoints. Provides `/ws/chat` for low-latency token streaming with cancellation signals and `/ws/documents/{doc_id}/progress` for live step-by-step document ingestion progress broadcasts.

• Called By: React UI (websocket.js) • Calls: ingest_worker.py, generator.py, pipeline.py, auth.py

**`backend/api/tools.py`** — AGENTIC TOOLS API

Endpoints for tool discovery ( `GET /api/tools` ), safe sandboxed tool execution ( `POST /api/tools/execute` ), and path-traversal-defended file downloads ( `GET /api/tools/download/{filename}` ) for generated Excel workbooks.

• Called By: React UI (ToolExecutionModal.jsx) • Calls: tools/registry.py, config.py

## 5.2 Background Workers & Asynchronous Job Layer

**`backend/workers/ingest_worker.py`** — ASYNCHRONOUS INGEST WORKER

Dedicated background ingestion worker. Runs format conversion, Docling vision extraction, PyMuPDF fallback, structure chunking, dense embeddings, Qdrant upserts, and BM25 index creation. Broadcasts real-time percentage updates via Redis pub/sub and WebSocket channels.

• Called By: job_queue.py / Docker CMD • Calls: converter.py, docling_parser.py, structure_chunker.py, embedder.py, vector_store.py, bm25.py

**`backend/workers/job_queue.py`** — JOB DISPATCHER & STATE TRACKER

Manages asynchronous job dispatching. Creates persistent `IngestJobModel` database records, tries Redis queue dispatch ( `docuchat_ingest_queue` ), and falls back gracefully to an in-memory async task pool when running in lightweight mode.

• Called By: api/documents.py • Calls: db/models.py, ingest_worker.py, redis.asyncio

## 5.3 Parsing, Chunking & Ingestion Layer

**`backend/parsers/converter.py`** — FORMAT NORMALIZER

Converts uploaded office documents (DOCX, PPTX, XLSX, TXT, HTML) and raster images (PNG, JPG) into standardized PDF representations using headless LibreOffice or PyMuPDF so the frontend canvas viewer can render them.

• Called By: ingest_worker.py • Calls: LibreOffice CLI, fitz (PyMuPDF)

**`backend/parsers/docling_parser.py`** — DOCLING VISION CLIENT

HTTP client for the Docling-serve microservice (Port 5001). Extracts document hierarchy, markdown tables, and bounding boxes. Implements an automatic local PyMuPDF fallback that extracts word-level rects and blocks if Docling is offline.

• Called By: ingest_worker.py • Calls: Docling REST API, fitz (PyMuPDF)

**`backend/chunking/structure_chunker.py`** — STRUCTURE CHUNKER

Decomposes parsed elements into token-budgeted chunks (~400 target tokens, 50 overlap). Preserves markdown heading paths ( `# Title > ## Section` ), table layouts, and associates raw sentence text with exact coordinate bounding boxes.

• Called By: ingest_worker.py • Calls: config.py, tiktoken

## 5.4 Embedding & Vector Storage Layer

**`backend/embeddings/embedder.py`** — EMBEDDING SERVICE

Unified embedding service supporting dual backends: Nomic Embed (768d via Ollama) and BGE-Small (384d via FastEmbed ONNX runtime / SentenceTransformers). Formats query search instructions and normalizes vectors.

• Called By: ingest_worker.py, pipeline.py • Calls: Ollama /api/embed, fastembed, sentence_transformers

**`backend/retrieval/vector_store.py`** — QDRANT CLIENT

Manages Qdrant Vector DB connection (Port 6333) with embedded storage fallback. Creates collection `doc_chunks` with Cosine distance, handles batch upserts with chunk metadata, and executes document-filtered ANN vector searches.

• Called By: main.py, ingest_worker.py, pipeline.py • Calls: qdrant_client

## 5.5 Retrieval, Fusion, Deduplication & Live Web Layer

**`backend/retrieval/bm25.py`** — LEXICAL BM25PLUS INDEX

Implements BM25Plus sparse lexical retrieval. Persists per-document inverted indexes to disk at `backend/data/bm25/{doc_id}.json` . Solves the standard BM25 negative score bug on long passages.

• Called By: ingest_worker.py, pipeline.py • Calls: rank_bm25, config.py

**`backend/retrieval/fusion.py`** — HYBRID RANK FUSION

Merges disparate ranked lists from vector ANN search and BM25Plus using Reciprocal Rank Fusion ($k=60$) and score-weighted fusion. Normalizes candidate scores into a unified distribution.

• Called By: pipeline.py • Calls: None (Pure math)

**`backend/retrieval/dedup.py`** — SEMANTIC DEDUPLICATOR

Filters redundant chunks by chunk ID, Jaccard word set similarity, and character n-gram overlap ($> 0.88$). Eliminates duplicate headers and sliding-window chunk overlaps.

• Called By: pipeline.py • Calls: None (Pure Python)

**`backend/retrieval/reranker.py`** — CROSS-ENCODER RERANKER

Executes deep cross-attention reranking with `BAAI/bge-reranker-base` . Uses **Max-Scaling Normalization** ( `raw / max_s` ) to prevent valid secondary candidate chunks from being dropped, filtering out candidates below 0.25 relevance.

• Called By: pipeline.py • Calls: sentence_transformers, CrossEncoder

**`backend/retrieval/pipeline.py`** — MASTER RETRIEVAL PIPELINE

Master orchestrator for the retrieval subsystem. Executes parallel multi-query vector and BM25 searches, passes hits through RRF fusion, applies deduplication, and invokes cross-encoder reranking to return the final Top 3 candidates.

• Called By: api/chat.py, api/websockets.py • Calls: vector_store.py, bm25.py, fusion.py, dedup.py, reranker.py

**`backend/retrieval/web_search.py`** — LIVE WEB ENGINE

Zero-API-key live web knowledge engine. Real-time market data via Yahoo Finance API, current events and movie lookups via DuckDuckGo Lite, and encyclopedic sports/history facts via Wikipedia REST API.

• Called By: llm/generator.py, tools/registry.py • Calls: httpx, requests

## 5.6 LLM Reasoning, Generation, LangChain & Agentic Tools Layer

**`backend/llm/client.py`** — OLLAMA CLIENT

Asynchronous HTTP client for Ollama (Port 11434). Handles raw model streaming, health checks, temperature adjustments, context window sizing (8192 tokens), and connection timeouts.

• Called By: rewriter.py, generator.py • Calls: httpx, config.py

**`backend/llm/router.py`** — QUERY ROUTER

Routes queries between `"document"` mode (grounded document facts, citations, PDF highlight) and `"general"` mode (ChatGPT real-world knowledge). Evaluates document presence and detects pleasantries/greetings.

• Called By: api/chat.py, api/websockets.py • Calls: config.py

**`backend/llm/rewriter.py`** — QUERY REWRITER

Uses chat history to resolve conversational pronouns (it, they, former, latter) and generates 3 query variants. Contains specialized fast expansions for tables, summaries, and financial reports.

• Called By: api/chat.py, api/websockets.py • Calls: llm/client.py

**`backend/llm/generator.py`** — GROUNDED GENERATOR

Assembles grounded prompts with numbered citations `[1]`. Enforces strict anti-hallucination rules. Streams tokens via SSE / WS. Runs `refine_citations_for_answer()` to isolate the single winning bounding box for PDF canvas highlighting.

• Called By: api/chat.py, api/websockets.py • Calls: client.py, langchain_adapter.py, web_search.py

**`backend/llm/langchain_adapter.py`** — LANGCHAIN ORCHESTRATOR

LangChain integration layer. Provides ChatOllama model orchestration, Document abstraction, ChatPromptTemplate formatting, and advanced PDF text cleaning ( `clean_pdf_text()` ) to fix hyphenated word breaks.

• Called By: generator.py • Calls: langchain_core, langchain_ollama, config.py

**`backend/tools/registry.py`** — SAFE TOOL REGISTRY

Central catalog of vetted agentic tools with Pydantic JSON schemas: `calculator` , `web_search` , and `export_excel` . Validates tool arguments and sandboxes execution with strict timeouts.

• Called By: api/tools.py • Calls: calculator.py, excel_generator.py, web_search.py

**`backend/tools/calculator.py`** — AST MATH EVALUATOR

Safe arithmetic engine using Python's `ast` parser. Evaluates additions, multiplications, powers, and trigonometry without ever executing `eval()` , preventing arbitrary code injection.

• Called By: tools/registry.py • Calls: ast, math

**`backend/tools/excel_generator.py`** — EXCEL GENERATOR

Automated styled spreadsheet generator via `openpyxl` . Builds styled headers, zebra-striped rows, auto-sized columns, and saves to `backend/data/output/` for secure client download.

• Called By: tools/registry.py • Calls: openpyxl, config.py

**`backend/eval/retrieval_eval.py`** — RAG EVALUATION SUITE

Evaluation harness for retrieval quality. Computes Hit Rate@K, Mean Reciprocal Rank (MRR@K), and Normalized Discounted Cumulative Gain (NDCG) across golden evaluation datasets.

• Standalone CLI • Calls: retrieval/pipeline.py, config.py

## 5.7 Database, Frontend & Orchestration Layer

**`backend/db/models.py`** — SQLALCHEMY 2.0 MODELS

Declarative ORM models: `UserModel` , `UserSessionModel` , `DocumentModel` , `IngestJobModel` , `ConversationModel` , and `MessageModel` . Supports multi-tenant foreign keys, JSON metadata, and cascading deletes.

• Called By: All DB services • Calls: sqlalchemy.orm

**`backend/db/session.py`** — DATABASE SESSION ENGINE

Manages async database engine and session factory ( `AsyncSessionLocal` ). Automatically manages PostgreSQL ( `asyncpg` ) connection pools and SQLite ( `aiosqlite` ) file locks.

• Called By: main.py, all API routers • Calls: sqlalchemy.ext.asyncio, config.py

**`backend/db/migrate_sqlite_to_pg.py`** — MIGRATION UTILITY

Automated schema and data migration utility. Reads tables and rows from local SQLite and inserts them into PostgreSQL with foreign key order preservation and conflict resolution.

• Standalone CLI • Calls: db/models.py, db/session.py

`backend/alembic/` **ALEMBIC SCHEMA MIGRATIONS**

Alembic migration environment. Version script `5d85f0b5984c_initial_schema_with_tenancy_and_jobs.py` establishes tables, tenancy indexes, and foreign keys across production databases.

• Standalone CLI (alembic upgrade head) • Calls: alembic.ini, db/models.py

`frontend/src/App.jsx` & `App.css` **REACT SPA & STATE CORE**

Main React application container. Manages application state (active document, conversation ID, settings, tool modals), orchestrates SSE streams and WebSocket events, and coordinates panel resizing.

• Mounted By: index.js • Calls: components/ Header, Chat, PdfViewer, Tools, Auth, Sidebar

`frontend/src/services/api.js` & `websocket.js` **CLIENT NETWORKING SERVICES**

Frontend networking layer. `api.js` handles Axios REST calls with automatic JWT headers. `websocket.js` manages resilient WebSocket connections, auto-reconnect, and progress callbacks.

• Called By: App.jsx, ChatArea.jsx • Calls: Native WebSocket API, axios

`frontend/src/components/PdfViewer/PdfViewerDrawer.jsx` **PDF CANVAS VIEWER**

PDF.js HTML5 canvas document renderer. Normalizes bounding box coordinates across varying PDF page zoom scales and renders the pulsing fluorescent highlight box directly over answer citations.

• Called By: App.jsx • Calls: pdfjs-dist

`docker-compose.yml` **MICROSERVICES ORCHESTRATOR**

Production Docker Compose specification defining all 8 services: `backend`, `worker`, `frontend`, `qdrant`, `docling`, `ollama`, `postgres`, and `redis`, with volume mounts, healthchecks, and restart policies.

• Executed By: docker compose up -d

# 6. Data Schemas, Resiliency & Quick Reference

## 6.1 Summary Matrix: What Server & Model Runs at Each Step

| PIPELINE STAGE | PRIMARY SOURCE FILE | CALLER → TARGET | MODEL / TECHNOLOGY | PRIMARY ARTIFACT / OUTPUT CREATED |
|---|---|---|---|---|
| 1. Upload | `api/documents.py` | React (3000) → FastAPI (5000) | HTTP Multipart POST | `backend/data/uploads/{id}_{file}` |
| 2. Job Enqueue | `workers/job_queue.py` | FastAPI → Redis (6379) | Redis RPUSH / Task Pool | `IngestJobModel(status="queued")` |
| 3. Format Conversion | `parsers/converter.py` | Worker Process → CLI | LibreOffice / PyMuPDF | `backend/data/converted/{id}.pdf` |
| 4. Layout Parsing | `parsers/docling_parser.py` | Worker → Docling (5001) | Docling / TableFormer / Fitz | `List[ParsedElement]` with exact BBoxes |
| 5. Token Chunking | `chunking/structure_chunker.py` | In-Process (Worker) | Tiktoken Structure Slicing | `List[Chunk]` (~400 tokens + bboxes) |
| 6. Embeddings | `embeddings/embedder.py` | Worker → Ollama / FastEmbed | Nomic 768d / BGE-small 384d | Normalized floating-point vectors |
| 7. Vector Index | `retrieval/vector_store.py` | Worker → Qdrant (6333) | Qdrant HNSW Graph Index | Points in collection `doc_chunks` |
| 8. BM25 Index | `retrieval/bm25.py` | In-Process (Worker) | BM25Plus Inverted Index | `backend/data/bm25/{doc_id}.json` |
| 9. User Query | `api/chat.py` / `websockets.py` | React (3000) → FastAPI (5000) | SSE Stream / WebSocket | User message record in database |
| 10. Tool Check | `tools/registry.py` | FastAPI → AST Sandbox | Safe AST / OpenPyXL / DDG | Calculated result or downloadable Excel |
| 11. Query Rewriting | `llm/rewriter.py` | FastAPI → Ollama (11434) | `qwen2.5:3b` / `7b` Model | 3 pronoun-resolved search queries |
| 12. Hybrid Search | `retrieval/pipeline.py` | FastAPI → Qdrant & BM25 | Cosine ANN + BM25 Lexical | Top 60 candidate chunks |
| 13. Fusion & Dedup | `retrieval/fusion.py` & `dedup.py` | In-Process | RRF ($k=60$) + Jaccard 3-Gram | Top 40 non-duplicate candidate chunks |
| 14. Cross-Rerank | `retrieval/reranker.py` | In-Process → PyTorch/ONNX | `BAAI/bge-reranker-base` | Top 3 chunks with score $\ge 0.25$ |
| 15. Generation | `llm/generator.py` | FastAPI → Ollama (11434) | `qwen2.5:3b` / `7b` via LangChain | Streamed token response + citations |
| 16. Highlight | `generator.py` & `PdfViewerDrawer.jsx` | FastAPI → React Canvas | `refine_citations_for_answer` | Pulsing fluorescent highlight on PDF.js |

### 6.2 Resiliency & Built-in Automatic Fallbacks

| COMPONENT / FAULT POINT | PRIMARY EXECUTION ENGINE | AUTOMATIC FALLBACK MECHANISM | USER / SYSTEM IMPACT |
|---|---|---|---|
| **Document Parser** | Docling-serve container (Port 5001) | Local `pymupdf` (Fitz) structural parser | Zero downtime. Automatically extracts text blocks and coordinates offline. |
| **Vector Database** | Qdrant Server (Port 6333) | Embedded local Qdrant ( `data/qdrant_storage` ) | Full vector search operates locally without Docker container. |
| **Relational Database** | PostgreSQL 16 (Port 5432) | SQLite local database ( `data/rag_app.db` ) | Complete persistence for users, jobs, and chat histories without Postgres. |
| **Job Message Broker** | Redis 7 Broker (Port 6379) | FastAPI asynchronous background task pool | Ingestion jobs execute seamlessly in-process with persistent DB tracking. |
| **Embeddings Engine** | Ollama ( `nomic-embed-text` ) | FastEmbed ONNX ( `bge-small-en-v1.5` ) | Sub-second local CPU embeddings if Ollama is busy or unreachable. |
| **Office File Converter** | Headless LibreOffice CLI | PyMuPDF text-canvas generation | Converts text and images to PDF without external office software. |
| **Password Hashing** | Argon2id ( `argon2_cffi` ) | PBKDF2-HMAC-SHA256 (100k rounds) | Zero dependency breaks; constant-time hash verification. |
| **Live Web Search** | Yahoo Finance & DuckDuckGo Lite | Wikipedia REST API | Factual lookups succeed without third-party API keys or rate limits. |

> **Production Readiness Certification:** DocuChat AI is architected with complete decoupled fault isolation. The entire RAG lifecycle can operate either as an enterprise 8-container Kubernetes/Docker cluster or as an air-gapped standalone developer workstation with local SQLite and FastEmbed ONNX.